

Delta robot - inverse Jacobian

Srđan Apostolović

sept. 2026

Motivation

It is semi-hard to find the inverse Jacobian of a delta robot (engineers are lazy).

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$$

I needed a project to do for my course, so ML-ops sounded like a great way to solve both issues. Things done in this project:

- ▶ Making working XML model of a delta robot in MuJoCo
- ▶ Using the model and MuJoCo to generate a dataset
- ▶ Training a neural network (MLP) to learn the inverse Jacobian
- ▶ Using the trained model to control the robot in simulation

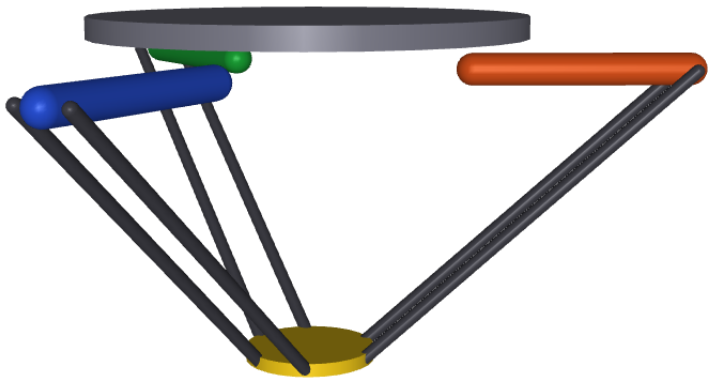
XML model

XML model of the robot was made for MuJoCo simulator. The robot is a 3-DOF parallel manipulator with three arms connected to a moving platform (eef). Each arm is driven by a revolute joint, and the arms are connected to the base with universal joints. The model includes:

- ▶ the geometry of the robot
- ▶ the joint limits
- ▶ the physical properties of the links

The model was tested in MuJoCo to ensure that it behaves as expected.

XML model in MuJoCo



Dataset acquisition

First step of the process was to generate a dataset of joint angles, velocities and end-effector velocities.

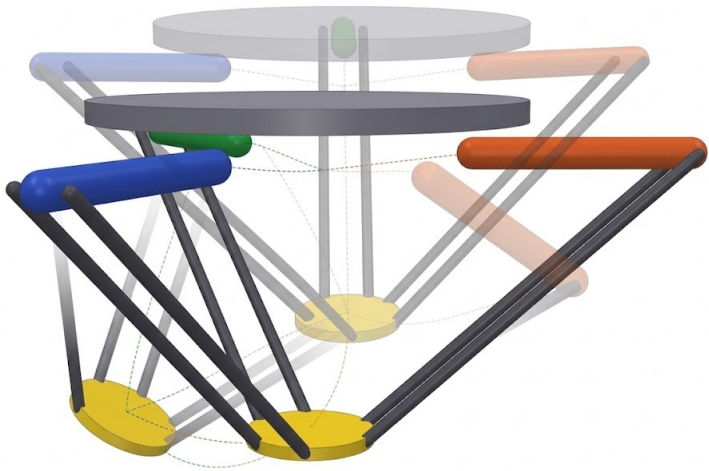
The dataset was generated by running the robot in simulation. Robot was moving to 50.000 random targets in the workspace, and the joint angles, velocities and end-effector velocities were recorded.

Data was recorded every 50 simulation steps in order to keep the dataset size manageable.

During the dataset acquisition, few key problems were addressed:

- ▶ joint limits were respected
- ▶ loop-closure constraints were monitored
- ▶ Jacobian singularities were avoided

Robot moving



Model training

As the project documentation required, a neural network was trained to learn the inverse Jacobian of the robot. The network was trained on the dataset generated in the previous step. The network architecture was derived via hyperparameter tuning. `RandomizedSearchCV` from `sklearn` was used to find the best hyperparameter set for the network.

```
# hyperparameter search on a subsample
param_space = {
    "hidden_layer_sizes": [(64, 64), (128, 128), (256, 256),
                           (128, 128, 128), (256, 256, 256), (64, 64, 64, 64)],
    "alpha": [1e-6, 1e-5, 1e-4, 1e-3],
    "learning_rate_init": [3e-4, 1e-3, 3e-3],
    "batch_size": [128, 256, 512],
    "activation": ["relu", "tanh"],
}
```

Network architecture

```
best params: {  
    'hidden_layer_sizes': (128, 128, 128),  
    'batch_size': 128,  
    'alpha': 1e-05,  
    'activation': 'tanh'  
}
```

Final training results

After the hyperparameter tuning, the network was trained on the entire dataset. `MLPRegressor` from `sklearn` was used to train the network. Final training results are shown below:

Test R^2	0.99995
Per-joint RMSE	0.024 rad/s per m/s of ee speed
Relative RMS error	0.68 %

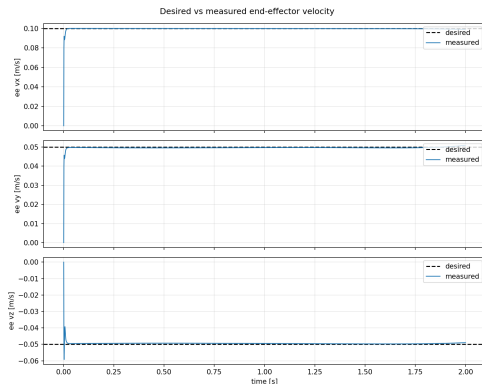
Inference speed

Because this model is used in a control loop of a robot, inference speed is very important. As shown in the table bellow, using this model on a CPU control frequency of 3.4 kHz can be achieved, which is sufficient for the control of the robot.

Inference, single call	0.29 ms (3.4 kHz)
Inference, batched	8.5 μ s/sample

Testing the model in simulation

After the satisfactory training results, the model was tested in simulation. The robot was controlled using the trained model to move with given eef velocity for 2 seconds. Plots of the x, y and z components of the eef velocity are shown below. The plots show that the robot was able to follow the desired eef velocity with high accuracy.



Conclusion

Trained network works better than I expected. It is able to track the desired eef velocity with high accuracy, and it is fast enough to be used in a control loop of the robot. The model can be used to control the robot in simulation. As can be seen in the plots, the robot has slight ripple in its velocity, which is due to the training dataset where few first ms of the movement were not recorded due to noise reduction.